

PROJECT DOCUMENTATION: MINI EMPLOYEE MANAGEMENT SYSTEM (EMS)

Prepared for: Final Internship Task – Project Delivery

Author: Internship Training Program Student

Technical Stack: React (Vite) | Node.js (Express) | Prisma ORM | PostgreSQL

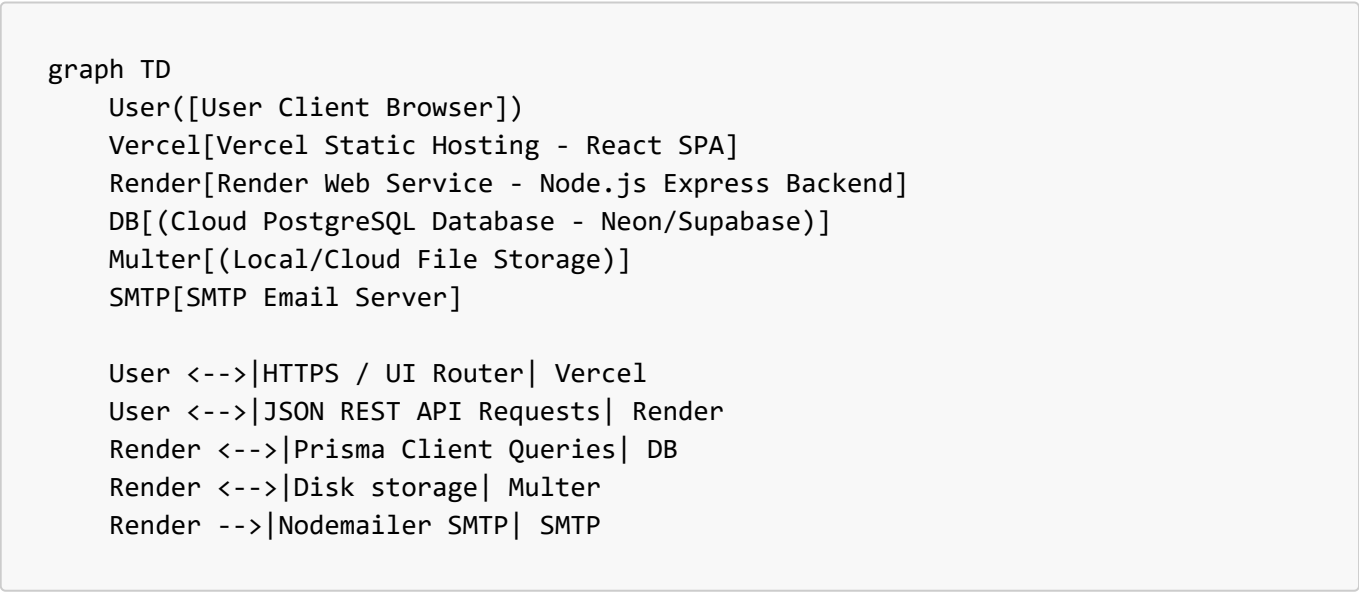
1. Executive Summary

The **Mini Employee Management System (EMS)** is a full-stack enterprise web application designed to streamline core human resource operations, including user administration, employee profiles, department allocation, skillset keywords tracking, multi-file attachments, automated welcome notifications, leave balance tracking, company asset allocation, and audit trails.

The application enforces role-based access control (RBAC), dividing users into **HR Administrators** (who manage registry, assets, leaves, and configurations) and **Employees** (who log shifts, request leaves, and receive notifications).

2. System Architecture

The application is structured as a decoupled full-stack architecture optimized for low-latency cloud environments:



- **Frontend SPA:** Built with React, Vite, Tailwind CSS, and Lucide React. Deployed on **Vercel** with custom client-side rewrite configurations (`vercel.json`) to handle seamless route refreshing.
- **Backend API Server:** Built with Express.js, Winston Logger, Joi Validators, and Swagger documentation. Deployed on **Render** utilizing automated health check hooks.

- **Database Layer:** Hosted on a cloud-managed **PostgreSQL** instance (Neon/Supabase), queried securely via **Prisma ORM** with structured transaction boundary handling.

3. Database Schema Specification

The database structure is designed in third normal form (3NF) to guarantee strict referential integrity. Explicit index patterns are added on foreign keys to optimize joins and search queries.

3.1 Entity Relationship Diagram

```
erDiagram
    User ||--o| Employee : "has profile"
    Department ||--o{ Employee : "contains"
    Employee ||--o{ EmployeeDocument : "uploads documents"
    Employee ||--o{ EmployeeSkill : "has skills"
    Skill ||--o{ EmployeeSkill : "assigned to"
    Employee ||--o{ LeaveBalance : "tracks days"
    LeaveType ||--o{ LeaveBalance : "defines limits"
    Employee ||--o{ LeaveApplication : "submits"
    LeaveType ||--o{ LeaveApplication : "categorized by"
    LeaveApplication ||--o{ ApprovalHistory : "logs review stages"
    Asset ||--o{ AssetAllocation : "allocated through"
    Employee ||--o{ AssetAllocation : "claims"
    Asset ||--o{ AssetHistory : "logs history"
    Employee ||--o{ Attendance : "logs shifts"
    User ||--o{ Notification : "receives"
    User ||--o{ AuditLog : "triggers activity"

    User {
        Int id PK
        String name
        String email UK
        String password
        String role
    }

    Employee {
        Int id PK
        Int user_id FK
        String first_name
        String last_name
        Int department_id FK
        String phone
        String address
        String designation
        Decimal salary
        DateTime created_at
    }

    EmployeeDocument {
        Int id PK
```

```
    Int employee_id FK
    String image_url
}

Skill {
    Int id PK
    String skill_name
}

EmployeeSkill {
    Int id PK
    Int employee_id FK
    Int skill_id FK
}

LeaveType {
    Int id PK
    String leave_name
    Int total_days
}

LeaveBalance {
    Int id PK
    Int employee_id FK
    Int leave_type_id FK
    Int available_days
}

LeaveApplication {
    Int id PK
    Int employee_id FK
    Int leave_type_id FK
    DateTime from_date
    DateTime to_date
    Int total_days
    String reason
    String status
    DateTime created_at
}

ApprovalHistory {
    Int id PK
    Int leave_id FK
    Int approved_by
    String action
    String remarks
    DateTime created_at
}

Asset {
    Int id PK
    String asset_code UK
    String asset_name
    String asset_type
}
```

```
        DateTime purchase_date
        Decimal purchase_cost
        String status
        DateTime created_at
    }

    AssetAllocation {
        Int id PK
        Int asset_id FK
        Int employee_id FK
        Int allocated_by
        DateTime allocated_date
        DateTime return_date
        String status
    }

    Attendance {
        Int id PK
        Int employee_id FK
        DateTime date
        DateTime clock_in
        DateTime clock_out
        String status
        Decimal total_hours
        String remarks
    }
}
```

3.2 Index Selections and Performance Optimizations

To ensure microsecond query execution and avoid full-table scans, the following indexes are applied:

Target Model	Index Definition	Primary Queries Optimized	Rationale
Employee	@@index([department_id])	Department listings, join queries.	Optimizes filtering employees by department and matching department names during JOINS.
EmployeeDocument	@@index([employee_id])	Fetching attachments/images.	Speeds up lookup of profile images for a specific employee.
EmployeeSkill	@@index([employee_id]), @@index([skill_id])	Many-to-many lookups.	Resolves join lookups between employees and skills rapidly.
LeaveApplication	@@index([employee_id]), @@index([status]), @@index([leave_type_id])	Leave histories, pending reviews, stats.	Optimizes dashboard lists (e.g., retrieving only "Pending" leaves).

Target Model	Index Definition	Primary Queries Optimized	Rationale
ApprovalHistory	<code>@@index([leave_id])</code>	Leave application audit trail.	Connects approval stages and remarks back to the parent leave request.
AssetAllocation	<code>@@index([employee_id]),</code> <code>@@index([asset_id]),</code> <code>@@index([status])</code>	Asset tracking, inventory reports, returns.	Speeds up checking active allocations or list of assets per employee.
Notification	<code>@@index([user_id]),</code> <code>@@index([is_read])</code>	Pulling unread alerts.	Crucial for the header alert counter which queries <code>is_read = false</code> on every page load.
AuditLog	<code>@@index([record_id]),</code> <code>@@index([table_name])</code>	Tracking history of specific resources.	Supports quick rendering of history trails for auditing.

4. API Specification

All endpoints are secured via JWT bearer tokens and require role-based access.

4.1 Authentication REST API

- **POST** `/api/v1/auth/register` (Public) - Create user account.
- **POST** `/api/v1/auth/login` (Public) - Authenticate and return Access & Refresh JWTs.

4.2 Employee Profiles (HR Role Required)

- **POST** `/api/v1/employees` - Register employee (creates User, Profile, maps Skills, and initializes Leave Balances). Triggers a welcome email.
- **GET** `/api/v1/employees` - Retrieve employees (supports pagination `?page=X&limit=Y` and fuzzy search `?search=Name`).
- **GET** `/api/v1/employees/:id` - Fetch detailed employee profile.
- **PUT** `/api/v1/employees/:id` - Update employee attributes.
- **DELETE** `/api/v1/employees/:id` - Delete employee profile and all dependent records.
- **POST** `/api/v1/employees/upload` - Attach up to 5 verified images/documents.

4.3 Leave Management Engine

- **GET** `/api/v1/leaves/types` - Get all leave categories (Casual, Sick, Annual).
- **GET** `/api/v1/leaves/balances?userId=X` - Retrieve available leave balances for a specific user.
- **POST** `/api/v1/leaves/apply` - Submit a leave request (validates balance availability).
- **GET** `/api/v1/leaves/pending` (HR Only) - List pending leave applications.

- **PATCH** `/api/v1/leaves/:id/hr-approve` (HR Only) - Approve leave request. Executed inside an **Atomic Transaction** (updates status, decrements leave balance, and creates approval history).
- **PATCH** `/api/v1/leaves/:id/hr-reject` (HR Only) - Reject leave request with remarks.

4.4 Asset Inventory Management

- **GET** `/api/v1/assets` - Get list of physical assets and status.
- **POST** `/api/v1/assets` (HR Only) - Register new physical asset.
- **POST** `/api/v1/assets/allocate` (HR Only) - Allocate asset code to an employee profile.
- **POST** `/api/v1/assets/return/:allocationId` (HR Only) - Mark asset as returned and set status to "Available".

4.5 Attendance Tracking (RBAC Shared)

- **POST** `/api/v1/attendance/clock-in` - Log shift start time (creates daily record).
- **POST** `/api/v1/attendance/clock-out` - Log shift end time (calculates total hours).
- **GET** `/api/v1/attendance/my-attendance?userId=X` - Retrieve personal monthly shifts.
- **GET** `/api/v1/attendance/all?date=YYYY-MM-DD` (HR Only) - Retrieve company-wide attendance for a specific day.

5. Setup & Production Deployment Guide

Phase 1: Deploying Cloud PostgreSQL Database (Neon.tech)

1. Sign up on [Neon.tech](#) and create a free project.
2. In the Neon dashboard, select your database project and copy the **PostgreSQL Connection String**.
3. Create a local backend `.env` file and set the `DATABASE_URL` variable:

```
DATABASE_URL="postgresql://username:password@hostname/neondb?
sslmode=require"
```

4. Push your schema layout directly to the Neon database by running the following command in the `/backend` folder:

```
npx prisma migrate deploy
```

5. Seed the database tables with default master data, test schedules, and pre-seeded HR/employee accounts:

```
node seed.js
```

Phase 2: Deploying Backend API Server to Render

1. Log in to [Render](#) and click **New > Web Service**.
2. Connect your GitHub repository containing the backend code.
3. Configure the service properties:
 - **Name:** `mini-ems-backend`
 - **Environment:** `Node`
 - **Build Command:** `npm install && npx prisma generate`
 - **Start Command:** `npm start`
4. Expand the **Advanced** section and define your Environment Variables:
 - `DATABASE_URL` = (Your Neon PostgreSQL Connection String)
 - `JWT_SECRET` = (A secure random hash key string)
 - `JWT_REFRESH_SECRET` = (A separate secure random hash key string)
 - `PORT` = `5000`
 - `EMAIL_USER` = (Your Gmail/SMTP address for notifications)
 - `EMAIL_APP_PASSWORD` = (Your Google App Mail Password)
5. Click **Create Web Service**. Once deployed, copy your Live Backend URL (e.g., <https://mini-ems-backend.onrender.com>).

Phase 3: Deploying Frontend to Vercel

1. Log in to [Vercel](#) and click **Add New > Project**.
2. Select your GitHub repository.
3. In the project build settings:
 - **Framework Preset:** `Vite`
 - **Root Directory:** `frontend`
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
4. Expand the **Environment Variables** panel and add:
 - `VITE_API_URL` = <https://mini-ems-backend.onrender.com> (Your live Render backend URL, without trailing slash)
5. Click **Deploy**. Vercel will build the React bundles and provide a live URL (e.g., <https://mini-ems-frontend.vercel.app>).

6. How to Run & Verify the Project Locally

If you want to run the full suite locally:

1. Clone the repository and navigate to the project directory.
2. **Launch Backend:**

```
cd backend
npm install
npx prisma generate
# Run migrations & seed data
npx prisma migrate dev --name init
node seed.js
# Start server
npm run dev
```

3. Launch Frontend:

```
cd ../frontend
npm install
# Start Vite dev server
npm run dev
```

4. Access the web app at <http://localhost:5173>.

5. Login Credentials:

- **HR Admin:** hr@hrms.com / HR@123
- **Employee:** aarnavyas495@gmail.com / Employee@123

7. Submission Checklist & Deliverables

To submit the project to the instructors, compile the following URLs:

1. **GitHub Repository Link:** <https://github.com/Atharva-28-04/Employee-Management-System.git>
2. **Live Frontend URL:** (E.g., <https://mini-ems-frontend.vercel.app>)
3. **Live Backend API URL:** (E.g., <https://mini-ems-backend.onrender.com/api/v1/health>)
4. **Cloud Database:** Hosted on Neon Cloud PostgreSQL.